By TOUCH DEN · November 11, 2026

Files and File Systems

Video 1 · Files, File Systems, and Paths

How Computers Work · Section 7

In this video, you will learn:

- What a file really is
- What a file system does
- Paths, directories, and the `..` trick

What a file really is

A file is just a named sequence of bytes on a drive.

A photo, a program, a video, a note, all the same to the computer: bytes with a name. What makes one a photo is only *how the bytes are read* (lesson 1 again).

- **Any file can be examined as raw bytes** (a hex dump), whatever its type.
- A file's "type" is interpretation, **not a deep property of the bytes** (it can be faked).

File systems

A drive is just a huge space for bytes. The **file system** organizes it (NTFS, APFS, ext4, FAT32).

It keeps:

- An **index** (directory) of every file: name, location, size.
- A map of **used vs free** space.
- Each file's **metadata** (permissions, timestamps).

Key idea: a file's name/index and its bytes are stored _separately_. (This unlocks deletion and recovery.)

Paths and the `..` trick

Files live in **directories** (folders), forming a tree. A **path** names a location:

- Linux/macOS: `/home/user/menu.txt` · Windows: `C:\Users\user\menu.txt`
- **Absolute** (from root) vs **relative** (from here). `.` = here, `..` = **up one level**.

> `..` is a security risk: a path like `../../etc/passwd` can climb **out** of the intended folder, **path traversal** (web attacks).

Paths from a user are untrusted input.

Recap: files, systems, paths

- A **file** is a named sequence of bytes; type is interpretation.
- A **file system** indexes files; name and bytes are stored **separately**.
- **Paths** name locations; `..` means up a level, the path-traversal trick.

Next video: metadata, magic bytes, and why ”deleted” isn’t gone.

Files and File Systems

Video 2 · Metadata, Magic Bytes, and Deletion

How Computers Work · Section 7

In this video, you will learn:

- File metadata and permissions
- Magic bytes, and why the extension lies
- Delete vs really delete, and mounting

Metadata and permissions

Every file carries **metadata**: size, timestamps, and **permissions**.

```
$ ls -l
-rw-r-----  user  staff  menu.txt
-rw-r--r--  user  staff  dot.png
```

Read `-rw-r-----` in groups: type, then **owner** (`rw-`), **group** (`r--`), **others** (`---`).

So `menu.txt` : owner read/write, group read, others nothing, least privilege, per file. Timestamps build a forensic **timeline**.

Magic bytes: the extension lies

How does a computer know it's a PNG? Not the extension, that's a changeable name.

Most formats start with a fixed signature, **magic bytes**:

```
$ xxd dot.png | head -1
00000000: 8950 4e47 0d0a 1a0a ... .PNG........
$ file menu.txt dot.png
menu.txt: ASCII text
dot.png:  PNG image data, 1 x 1, ...
```

`89 50 4e 47` = PNG, whatever the file is named. **Attackers fake extensions to hide malware; trust the content, not the name.**

Delete vs really delete

Delete a file + empty trash: is it gone? **Usually not.**

Name/index and bytes are **separate**, so delete just **removes the index entry and frees the space**. The bytes stay until overwritten.

> Like removing a book's catalogue card but leaving the book on the shelf.

- **Deleted files are often recoverable** (file carving, forensics).
- **Really erasing** needs **overwriting**, encryption, or physical destruction.

Mounting: connecting a device's file system into the tree. Forensics mounts **read-only** to preserve evidence.

Recap: metadata, magic, deletion

- **Metadata** (permissions `rwx`, timestamps) controls access and builds timelines.
- **Magic bytes** reveal the real type; the **extension can lie**, trust content.
- **Delete** removes the index (bytes remain, recoverable); **wipe** overwrites.
- Forensics mounts drives **read-only**, never alter the original.

Next lesson: a special kind of file, the program, and how code becomes something the CPU runs.